Universidad Autónoma del Estado de Hidalgo
Licenciatura en Ciencias Computacionales
Automatas y Compiladores
Docente: Dr.Eduardo Cornejo Velazquez

REPORTE DE PRÁCTICA NO.6:

Análisis sintáctico de expresiones aritméticas

ALUMNO:
Naomi Romero López
Ángel Ortega Gómez
Adrian Hernández Sánchez
Luis Jose Moreno Ortega

1. Introducción

En el ámbito del desarrollo de software y la ingeniería de sistemas, la capacidad de una computadora para interpretar y procesar lenguajes de programación o expresiones matemáticas no es un proceso trivial. Este proceso se fundamenta en la teoría de lenguajes formales y autómatas, la cual permite traducir cadenas de texto en instrucciones ejecutables mediante una estructura lógica y jerárquica.

La construcción de un compilador o intérprete se divide en etapas críticas que garantizan que la entrada sea válida tanto en su forma como en su significado. Las dos primeras fases, el análisis léxico y el análisis sintáctico, son los pilares de este proyecto. Mientras que el primero se encarga de identificar las unidades mínimas de información (tokens), el segundo asegura que dichas unidades respeten las reglas gramaticales preestablecidas, como la precedencia de operadores y el uso de paréntesis en expresiones aritméticas.

La presente práctica tiene como objetivo implementar un sistema de reconocimiento de expresiones utilizando Lex y Yacc, herramientas estándar en la industria para la generación automática de analizadores. A través de este desarrollo, se busca comprender cómo la teoría de gramáticas formales se materializa en herramientas funcionales capaces de validar operaciones complejas y gestionar la jerarquía de los datos de manera eficiente.

2. Marco teórico

Gramáticas formales

Una gramática formal es un modelo matemático que permite describir la estructura de un lenguaje mediante reglas bien definidas.

Se representa como una tupla $G = (\Sigma_T, \Sigma_N, S, P)$, donde Σ_T corresponde a los símbolos terminales, Σ_N a los símbolos no terminales, S es el símbolo inicial y P es el conjunto de producciones.

Estas reglas permiten generar cadenas válidas a partir del símbolo inicial, constituyendo la base para el análisis sintáctico en los lenguajes de programación [1].

Expresiones aritméticas

Las expresiones aritméticas son combinaciones de operandos y operadores que siguen una estructura definida por reglas gramaticales.

Estas expresiones incluyen números, operadores como suma, resta, multiplicación y división, así como el uso de paréntesis para modificar la precedencia.

La correcta interpretación depende de reglas como la prioridad de operadores, donde la multiplicación y división se evalúan antes que la suma y resta [4].

Analizador léxico

El analizador léxico es la primera etapa en el procesamiento de una cadena de entrada.

Su función es identificar y agrupar los caracteres en unidades significativas llamadas tokens. Este proceso simplifica el análisis posterior al transformar la entrada en una secuencia estructurada que puede ser interpretada por el analizador sintáctico.

Herramientas como Lex permiten automatizar este proceso mediante el uso de expresiones regulares [3].

Analizador sintáctico

El analizador sintáctico es el encargado de verificar si la secuencia de tokens cumple con las reglas de una gramática formal.

Este componente determina la estructura de la cadena de entrada y permite detectar errores sintácticos.

Además, construye representaciones como árboles sintácticos que reflejan la jerarquía de las operaciones en una expresión [2].

Árbol sintáctico

El árbol sintáctico es una representación jerárquica de una expresión donde los nodos internos corresponden a operadores y las hojas a operandos.

Este tipo de estructura permite visualizar el orden en que se deben evaluar las operaciones, respetando la precedencia y asociatividad.

Por ejemplo, en una expresión aritmética, la multiplicación se representa en un nivel inferior al de la suma debido a su mayor prioridad [3].

Herramientas Lex y Yacc

Lex y Yacc son herramientas fundamentales en la construcción de compiladores.

Lex se utiliza para generar analizadores léxicos a partir de patrones definidos mediante expresiones regulares, mientras que Yacc se encarga de generar analizadores sintácticos a partir de gramáticas formales.

La combinación de ambas herramientas permite reconocer, validar y procesar expresiones de manera automática [3]

Reconocimiento de expresiones

El reconocimiento de expresiones consiste en analizar una cadena de entrada para determinar si pertenece a un lenguaje definido por una gramática.

Este proceso implica la identificación de tokens, la aplicación de reglas de producción y la validación de la estructura completa de la expresión.

Si la cadena cumple con las reglas establecidas, se considera válida; en caso contrario, se detecta un error sintáctico [2].

3. Herramientas empleadas

Para esta práctica fueron utilizados los siguientes recursos de software:

1. **Flex (Fast Lexical Analyzer Generator):** Herramienta empleada para la generación del analizador léxico mediante el uso de expresiones regulares.
2. **Bison / Yacc (Yet Another Compiler-Compiler):** Generador de analizadores sintácticos encargado de validar la gramática de las expresiones.
3. **GCC (GNU Compiler Collection):** Compilador de lenguaje C utilizado para la creación del archivo ejecutable final.
4. **Ambiente de desarrollo FLEX Windows:** Entorno integrado que facilita la compilación y enlace (linking) de los componentes léxicos y sintácticos.

4. Objetivos

Objetivo general

Desarrollar un analizador léxico y un analizador sintáctico utilizando Lex y Yacc para reconocer y validar operaciones aritméticas, asegurando la detección de errores y la correcta validación de las gramáticas definidas.

Objetivos específicos

- Emplear las herramientas conceptuales (autómatas y gramáticas libres de contexto) para definir cómo lograr el objetivo de la práctica.
- Diseñar un programa en Lex que identifique números enteros y números decimales, así como símbolos que representan las operaciones de suma, resta, multiplicación y división en el lenguaje de programación Python.
- Implementar un programa en Yacc que verifique la correcta construcción de las operaciones aritméticas de suma, resta, multiplicación y división mediante una gramática formal.

5. Desarrollo

Para la implementación del Autómata Finito Determinista (AFD) se utilizó el lenguaje de programación Java. El programa fue diseñado para leer la información del autómata desde un archivo de texto y posteriormente evaluar un conjunto de cadenas para determinar si son aceptadas o rechazadas por el autómata definido.

Diseño de Autómatas Finitos para el Reconocimiento de Tokens

En esta sección se describen los modelos formales para cada token del lenguaje. Cada autómata se define por su quíntupla $M = (Q, \Sigma, \delta, q_0, F)$ y su comportamiento de transición.

A. Números Enteros

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

Donde:

- $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- $Q = \{q0, q1\}$
- $q_0 = q0$
- $F = \{q1\}$

Función de transición

$$\delta(q0, [0-9]) = q1$$
$$\delta(q1, [0-9]) = q1$$

Tabla de transición

Estado	0-9
→ q0	q1
* q1	q1

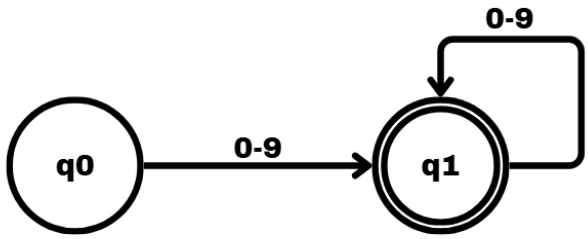

Figure 1: Diagrama de transición de estados de números enteros

B. Números Decimales

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

Donde:

- $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, .\}$
- $Q = \{q0, q1, q2, q3\}$
- $q_0 = q0$
- $F = \{q3\}$

Función de transición

$$\begin{aligned} \delta(q0, [0-9]) &= q1 \\ \delta(q1, [0-9]) &= q1 \\ \delta(q1, .) &= q2 \\ \delta(q2, [0-9]) &= q3 \\ \delta(q3, [0-9]) &= q3 \end{aligned}$$

Tabla de transición

Estado	0-9	.
→ q0	q1	-
q1	q1	q2
q2	q3	-
* q3	q3	-

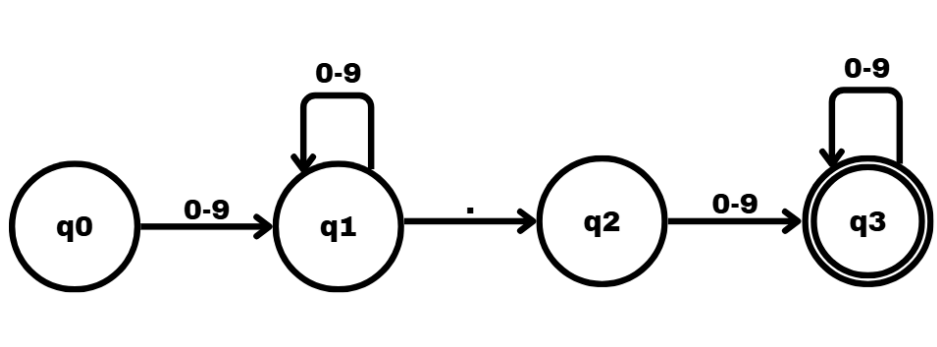

Figure 2: Diagrama de transición de estados de números decimales

C. Identificadores

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

Donde:

- $\Sigma = \{a\text{-}z, A\text{-}Z, _, 0\text{-}9\}$
- $Q = \{q0, q1\}$
- $q_0 = q0$
- $F = \{q1\}$

Función de transición

$$\delta(q0, [a\text{-}zA\text{-}Z_]) = q1$$
$$\delta(q1, [a\text{-}zA\text{-}Z_0\text{-}9]) = q1$$

Tabla de transición

Estado	a-z, A-Z, _	0-9
→ q0	q1	-
* q1	q1	q1

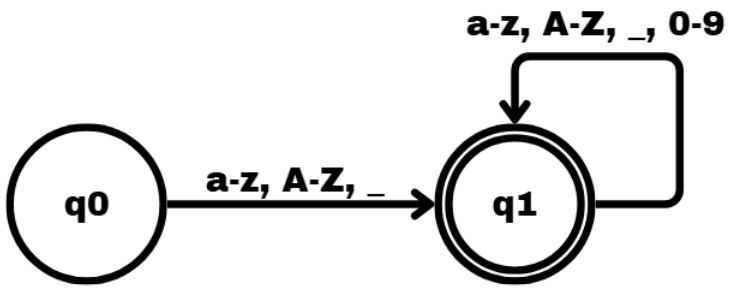

Figure 3: Diagrama de transición de estados para identificadores

D. Operadores Aritméticos

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

Donde:

- $\Sigma = \{+, -, *, /\}$
- $Q = \{q0, q1, q2, q3, q4\}$
- $q_0 = q0$
- $F = \{q1, q2, q3, q4\}$

Función de transición

$$\delta(q0, +) = q1$$
$$\delta(q0, -) = q2$$
$$\delta(q0, *) = q3$$
$$\delta(q0, /) = q4$$

Tabla de transición

Estado	+	-	*	/
→ q0	q1	q2	q3	q4
* q1	-	-	-	-
* q2	-	-	-	-
* q3	-	-	-	-
* q4	-	-	-	-

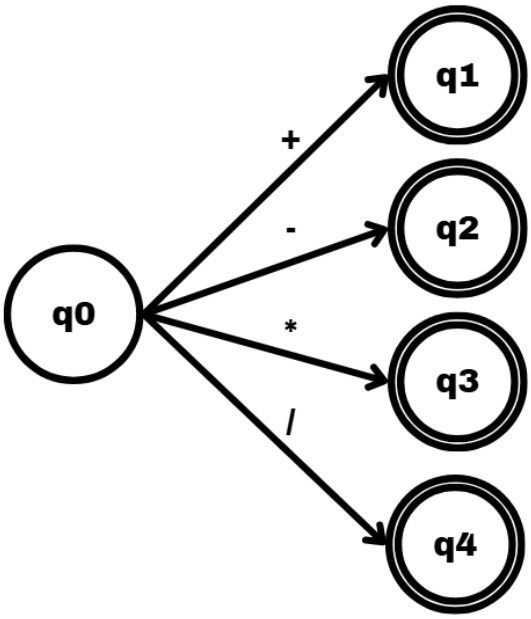

Figure 4: Diagrama de transición de estados para operadores aritméticos

E. Operadores Relacionales

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

Donde:

- $\Sigma = \{>, <, =, !\}$
- $Q = \{q0, q1, q2, q3, q4, q5, q6, q7, q8\}$

- $q_0 = q0$
- $F = \{q1, q2, q3, q4, q6, q8\}$

Función de transición

$$\delta(q0, >) = q1$$
$$\delta(q1, =) = q2$$

$$\delta(q0, <) = q3$$
$$\delta(q3, =) = q4$$

$$\delta(q0, =) = q5$$
$$\delta(q5, =) = q6$$

$$\delta(q0, !) = q7$$
$$\delta(q7, =) = q8$$

Tabla de transición

Estado	>	<	=	!
→ q0	q1	q3	q5	q7
* q1	-	-	q2	-
* q2	-	-	-	-
* q3	-	-	q4	-
* q4	-	-	-	-
q5	-	-	q6	-
* q6	-	-	-	-
q7	-	-	q8	-
* q8	-	-	-	-

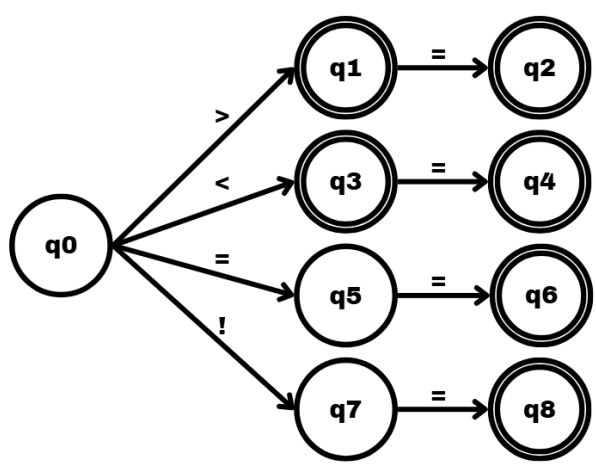

Figure 5: Diagrama de transición de estados para operadores relacionales

F. Paréntesis

$$AFD = (\Sigma, Q, \delta, q_0, F)$$

Donde:

- $\Sigma = \{(,)\}$
- $Q = \{q0, q1, q2\}$
- $q_0 = q0$
- $F = \{q1, q2\}$

Función de transición

$$\delta(q0, () = q1$$
$$\delta(q0,)) = q2$$

Tabla de transición

Estado	(	)
→ q0	q1	q2
* q1	-	-
* q2	-	-

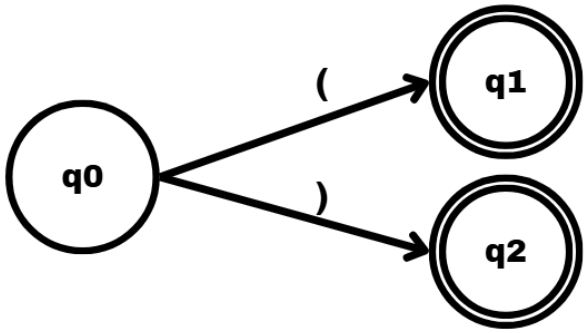

Figure 6: Diagrama de transición de estados para paréntesis

Notación BNF para describir la gramática de las operaciones

Para la validación sintáctica de las operaciones aritméticas, se ha definido la siguiente gramática en notación Backus-Naur (BNF), la cual establece la jerarquía de operadores y la recursividad de las expresiones:

```
<expresion>     ::= <expresion> "+" <termino>
                  | <expresion> "-" <termino>
                  | <termino>

<termino>       ::= <termino> "*" <factor>
                  | <termino> "/" <factor>
                  | <factor>

<factor>        ::= "(" <expresion> ")"
                  | <numero>
                  | <decimal>
                  | <identificador>

<numero>        ::= <digito> | <digito> <numero>

<decimal>       ::= <numero> "." <numero>

<identificador> ::= <letra> | <letra> <identificador> | <letra> <digito>
```

Definiciones de Símbolos Terminales

- <**digito**> ::= "0" — "1" — "2" — "3" — "4" — "5" — "6" — "7" — "8" — "9"
- <**letra**> ::= "a" — "b" — ... — "z" — "A" — ... — "Z" — "_"

Desarrollo del programa en Lex para reconocer los tokens:

• Números enteros.

• Números decimales.

• Símbolos que representan las operaciones de suma, resta, multiplicación y división en el lenguaje de programación Python.

• Símbolos para construir las expresiones condicionales (por ejemplo, comparaciones).

• Identificadores (Nombres de variables)

• Paréntesis

```
%{
#include "y.tab.h"
#include <stdio.h>
#include <stdlib.h>
%}

%%

[0-9]+\.[0-9]+      { yylval.floatval = atof(yytext); return DECIMAL; }
[0-9]+              { yylval.intval = atoi(yytext); return NUM; }
```

```
"+"                 { return PLUS; }
"-"                 { return MINUS; }
"*"                 { return MULT; }
"/"                 { return DIV; }

">="                { return MAYOR_IGUAL; }
"<="                { return MENOR_IGUAL; }
"=="                { return IGUAL; }
"!="                { return DIFERENTE; }
">"                 { return MAYOR; }
"<"                 { return MENOR; }

"("                 { return PAR_IZQ; }
")"                 { return PAR_DER; }

[a-zA-Z_][a-zA-Z0-9_]* { return ID; }

\n                  { return EOL; }
[ \t\r]+            ;
.                   { printf("Error lexico: %s\n", yytext); }

%%

int yywrap(void) { return 1; }
```

Desarrollo del programa en Yacc

•Verificar que las operaciones aritméticas planteadas cumplan las reglas definidas.

• Agregar acciones que confirmen si la expresión es válida o no.

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
int yyerror(const char *s);
%}

%union {
    int intval;
    float floatval;
    char *stringval;
}

%token <intval> NUM
%token <floatval> DECIMAL
%token <stringval> ID
%token NUMBER PLUS MINUS MULT DIV EOL
%token PAR_IZQ PAR_DER
```

```
%start statements

%left PLUS MINUS
%left MULT DIV

//NO se usan en operaciones aritmeticas
%token MAYOR_IGUAL MENOR_IGUAL IGUAL DIFERENTE MAYOR MENOR

%%

statements
    : statement statements
    | statement
    ;

statement
    : expression EOL { printf("Expresion valida\n"); }
    | error EOL { printf("Expresion invalida\n"); yyerrok; }
    ;

expression
    : NUM
    | DECIMAL
    | ID
    | expression PLUS expression
    | expression MINUS expression
    | expression MULT expression
    | expression DIV expression
    | PAR_IZQ expression PAR_DER
    ;

%%

extern FILE *yyin;

int main(){
    printf("Practica 6: Analisis sintactico expresiones aritmeticas\n\n");

    printf("=== ARCHIVO CORRECTAS ===\n");
    yyin = fopen("D:\\Flex Windows\\EditPlusPortable\\ExpA_Correctas.txt", "r");

    if(yyin == NULL){
        printf("No se pudo abrir correctas.txt\n");
 system("pause");
        return 1;
    }

    yyparse();
    fclose(yyin);

    printf("\n=== ARCHIVO INCORRECTAS ===\n");
    yyin = fopen("D:\\Flex Windows\\EditPlusPortable\\ExpA_Incorrectas.txt", "r");

    if(yyin == NULL){
```

```
        printf("No se pudo abrir incorrectas.txt\n");
 system("pause");
        return 1;
    }

    yyparse();
    fclose(yyin);

    system("pause");
    return 0;
}

int yyerror(const char *s){
    return 0;
}
```

Archivos de texto con entradas correctas e incorrectas

```
Correctas:
2+3
(4+5)*6
10/2+7
8*(3+2)-4
(1+2)*(3+4)/5

Incorrectas:
2+
(3*4
5+/2
*7+3
(8+2))
```

Archivos de texto con entradas correctas e incorrectas

El desarrollo de un compilador (o intérprete) se divide en etapas donde los archivos se transforman sucesivamente. A continuación, detallamos el flujo de trabajo seguido en el ambiente FLEX Windows:

Vinculación mediante y.tab.h

El archivo y.tab.h se puede entender como un puente de comunicación. Se genera al compilar el archivo de Yacc y contiene las definiciones numéricas de los tokens.

En Lex este se incluye con #include ”y.tab.h” para que el analizador léxico sepa qué código entregarle al sintáctico cuando encuentra una coincidencia.

Su importancia radica en que sin este archivo, Lex no reconocería las palabras clave que Yacc necesita para validar la gramática.

Flujo de los botones en el IDE

El proceso de construcción se realizó siguiendo una jerarquía de dependencias:

1. Yacc Compile (Botón 3): Toma el archivo .y y genera dos archivos en lenguaje C: y.tab.c (el analizador sintáctico) y y.tab.h (las definiciones de tokens).
2. Lex File Compile (Botón 1): Toma el archivo .l y, utilizando el encabezado generado en el paso anterior, crea el archivo lex.yy.c. Este archivo contiene la función yylex(), encargada de convertir el texto de entrada en una secuencia de tokens.
3. Lex+Yacc Build (Botón 4): Esta es la etapa de Enlace (Linking). El compilador de C (GCC) toma tanto a lex.yy.c como a y.tab.c, los combina y genera un único archivo ejecutable (.exe). Aquí se resuelven las referencias cruzadas entre ambas partes del programa.
4. Execute exe (Botón 5): Lanza el programa final en la consola para realizar las pruebas de validación de expresiones.

6. Resultados

En esta sección se presentan los resultados obtenidos tras la ejecución del analizador léxico y sintáctico desarrollado con Lex y Yacc.

Se procesaron dos conjuntos de pruebas: uno compuesto por expresiones correctas y otro por expresiones con errores sintácticos, con el objetivo de validar el funcionamiento del sistema.

En la Figura 7 se observan los archivos ingresados asi como los resultados obtenidos correspondientemente

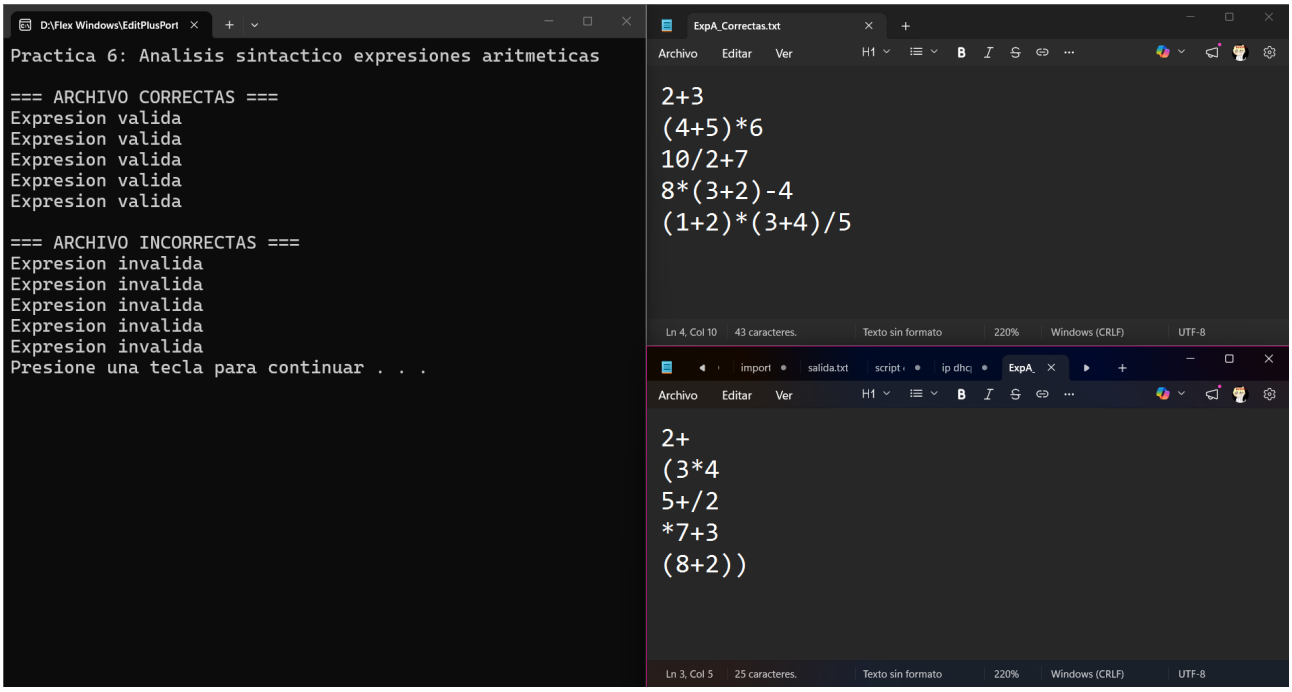

Figure 7: Resultados

7. Cuestionario

Pregunta 1: ¿Qué tipo de errores fueron detectados en los ejemplos incorrectos y cómo influyeron las gramáticas definidas para lograr su identificación?

Respuesta: Los errores detectados en las pruebas fueron principalmente de sintaxis, los cuales ocurren cuando el orden de los elementos ingresados no coincide con las reglas estructurales del lenguaje. Por ejemplo, al ingresar una expresión donde faltan operadores o sobran paréntesis, el analizador detecta que la secuencia de tokens no es válida.

La gramática influye directamente al funcionar como un mapa de rutas permitidas. Cuando el usuario ingresa algo incorrecto, el autómata de Yacc llega a un estado donde no hay una transición posible para el siguiente token. Gracias a la regla de error que definimos, el programa identifica el punto exacto de la falla y emite una alerta sin detener su ejecución, permitiendo al usuario corregir la entrada.

Pregunta 2: ¿Cómo contribuye el uso de derivaciones por la izquierda o por la derecha en la construcción de un árbol sintáctico para validar las opraciones y las prioridades de las operaciones?

Respuesta: Las derivaciones por la izquierda o derecha determinan cómo se construye el árbol sintáctico que representa la operación. En el caso de Yacc, que utiliza un análisis ascendente, el árbol se edifica desde los operandos hacia la raíz, asegurando que las operaciones con mayor prioridad se agrupen en la parte más profunda de la estructura.

Esta organización jerárquica es la que permite validar que una multiplicación se realice antes que una suma. Al construir el árbol siguiendo estas reglas, el compilador garantiza que el orden de ejecución respete las leyes de precedencia matemática, otorgando un significado lógico y estructurado a la cadena de texto original.

Pregunta 3: ¿Qué ajustes serían necesarios en la gramática para permitir el reconocimiento de estructuras condicionales más complejas, como $(a + b)^2$, raiz cuadradada, así como operaciones trigonométricas?

Respuesta: Para permitir el uso de potencias o funciones como la raíz cuadrada y el seno, el primer ajuste necesario sería en el analizador léxico. Se deben crear tokens específicos para símbolos como ŷ para palabras reservadas como sqrt o sin, permitiendo que el programa los identifique como unidades con significado especial.

En la gramática de Yacc, se tendrían que agregar nuevas reglas de producción que definan cómo deben aparecer estos términos. Por ejemplo, se establecería que una función trigonométrica debe ir seguida obligatoriamente por un paréntesis y una expresión. Además, se incluiría la librería `<math.h>` en el código C para que el programa pueda calcular los resultados matemáticos de dichas funciones durante la ejecución.

8. Conclusiones

El desarrollo de este proyecto permite concluir que la integración de Lex y Yacc constituye una metodología robusta y eficiente para el procesamiento de lenguajes. A través de la implementación realizada, se observó que la separación de responsabilidades entre el análisis léxico y el sintáctico no solo simplifica el diseño del software, sino que garantiza un control riguroso sobre la estructura de la entrada.

Uno de los hallazgos más relevantes es la importancia de las gramáticas formales en la definición de la precedencia de operaciones. El uso de un árbol sintáctico no es meramente una representación visual, sino la base lógica que permite que una computadora comprenda que, en una expresión aritmética, una multiplicación debe ejecutarse con prioridad sobre una suma, independientemente del orden de aparición.

Asimismo, la práctica demuestra que el uso de herramientas de automatización para compiladores reduce significativamente el margen de error en comparación con el análisis manual de cadenas. La capacidad de detectar y reportar errores sintácticos de forma precisa reafirma que el estudio de los traductores y compiladores es esencial para cualquier desarrollo tecnológico que requiera una comunicación estructurada y confiable entre el usuario y la máquina.

Finalmente, la realización de este trabajo permite valorar la relevancia académica y profesional de estas herramientas en la formación de un ingeniero, ya que trasciende la simple validación de expresiones para adentrarse en la comprensión profunda de la abstracción computacional. El dominio de Lex y Yacc no solo facilita la creación de lenguajes de dominio específico (DSL), sino que también agudiza la capacidad de resolución de problemas al obligar al desarrollador a pensar de manera estructurada y lógica, sentando las bases para explorar fases más avanzadas como la optimización de código y la gestión de memoria.

Referencias Bibliográficas

References

[1] Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación*. Pearson Educación.

[2] Gutú, O. (2013). *Primer curso en teoría de autómatas y lenguajes formales*. Pearson Educación.

[3] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compiladores: principios, técnicas y herramientas*. Pearson Educación.

[4] Sebesta, R. W. (2010). *Conceptos de lenguajes de programación*. Pearson Educación.